LAPORAN PRAKTIKUM
STRUKTUR DATA
ABSTRACT DATA TYPE JAM

Disusun oleh:

Ghinada Fathanawafa Algma
2511533008

Dosen Pengampu:
Dr. Wahyudi S.T. M.T

Asisten Praktikum:
Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadirat Tuhan Yang Maha Esa, berkat Rahmat dan karunia-Nya laporan praktikum Struktur Data tentang ADT Jam menggunakan bahasa Java ini dapat saya selesaikan dengan baik. Laporan ini saya susun sebagai bentuk pertanggungjawaban atas praktikum yang telah saya laksanakan.

Dalam penyusunan laporan ini saya banyak mendapat bimbingan serta dukungan dari beberapa pihak, oleh karena itu saya mengucapkan terima kasih kepada dosen pengampu, asisten praktikum dan rekan rekan praktikan yang turut membantu dalam pelaksanaan kegiatan praktikum.

Saya menyadari bahwa lapporan praktikum ini jauh dari sempurna. Oleh karena itu kritik dan saran yang membangun sangat berguna untuk perkembangan lebih baik kedepannya. Semoga laporan praktikum ini dapat bermanfaat bagi saya serta pembaca, khususnya dalam memahami ADT Jam.

Padang, Maret 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan 1

1.3 Manfaat 1

BAB II PEMBAHASAN 2

2.1 Dasar Teori 2

2.2 Langkah Kerja 3

2.2.1 Membuat ADT Jam 3

2.2.2 Membuat Jam Driver 1 6

2.2.3 Membuat Jam Driver 2 7

2.3 Output program 8

2.3.1 Output Jam Driver 8

2.3.2 Output Jam Driver 2 9

BAB III PENUTUP 10

3.1 Kesimpulan 10

DAFTAR PUSTAKA 11

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam mata kuliah Struktur Data, salah satu tujuan utama adalah memahami bagaimana suatu data dapat direpresentasikan secara abstrak melalui Abstract Data Type (ADT). ADT Jam merupakan salah satu contoh penerapan konsep ini, di mana waktu (jam, menit, detik) direpresentasikan sebagai sebuah tipe data abstrak dengan operasi-operasi tertentu.

Dengan adanya praktikum ADT Jam, mahasiswa diharapkan mampu memahami konsep abstraksi data, menguasai implementasi struktur data sederhana, serta mengembangkan keterampilan berpikir logis dan sistematis dalam pemrograman.

1.2 Tujuan

1. Memahami konsep ADT (Abstract Data Type).
2. Mempelajari perancangan struktur data sederhana.
3. Mengimplementasikan operasi dasar pada ADT Jam.

1.3 Manfaat

1. Pemahaman terhadap konsep abstraksi.
2. Dapat merancang struktur data sederhana.
3. Dapat mengimplementasikan operasi dasar yang ada pada ADT Jam.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Abstract Data Type (ADT)

ADT merupakan konsep dalam pemrograman yang menggambarkan sebuah tipe data dan operasi operasi yang dapat dilakukan terhadap tipe data tersebut tanpa harus mengungkapkan detail implementasinya.

ADT biasanya diimplementasikan dalam 2 modul

1. Definisi/Spesifikasi dari TYPE dan PRIMITIF
    - Spesifikasi TYPE sesuai bahasa
    - Spesifikasi PRIMITIF sesuai konteks (fungsi dan prosedur)
2. Body, berupa kode program

Realisasinya dalam bahasa pemrograman

Java → Spesifikasi : Class → body : Public Class

Primitif dikelompokkan sebagai berikut :

1. Konstruktor/Kreator, pembentuk nilai type. Semua objek (variabel) ber-type tersebut harus melalui konstruktor. Biasanya namanya diawali Make.
2. Selektor, untuk mengakses komponen type (biasanya namanya diawali dengan Get).
3. Prosedur pengubah nilai komponen (biasanya namanya diawali Get).
4. Validator komponen type, yang dipakai untuk mentest apakah dapat membentuk type sesuai dengan batasan.
5. Destruktor/Dealokator, yaitu untuk “menghancurkan” nilai objek (sekaligus memori penyimpannya).
6. Baca/Tulis, untuk interface dengan input/output device.
7. Operator relational, terhadap type tersebut untuk mendefinisikan lebih besar, lebih kecil, sama dengan, dan sebagainya.

8. Aritmatika terhadap type tersebut karena biasanya aritmatika dalam bahasa pemrograman hanya terdefinisi untuk bilangan numerik.
9. Konversi dari type tersebut ke type dasar dan sebaliknya.

2.2 Langkah Kerja

1. Buat Project baru dengan nama PrakSD2026_B_2511533008
2. Buat Package baru dalam project tersebut dengan nama pekan1_2511533008

2.2.1 Membuat ADT Jam

1. Buat class baru dalam package yang dibuat dengan nama Jam_2511533008.
2. Buat kode program berikut :

```java
package pekan1_2511533008;

public final class Jam_2511533008 {
    private int hh; // 0...23
    private int mm; // 0...59
    private int ss; // 0...59

    //---Validator---
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
    }

    //---Konstruktor/Kreator---
    public Jam_2511533008(int h, int m, int s) {
        this.hh = h; this.mm = m; this.ss = s;
    }

    //---Selektor---
    public int getHour() {return hh;}
    public int getMinute() {return mm;}
    public int getSecond() {return ss;}

    //---Mutator---
    public void setHour(int h) {this.hh = h; }
    public void setMinute(int m) {this.mm = m;}
    public void setSecond(int s) {this.ss = s;}

    //---Konversi---
    public int toSeconds() { return hh * 3600 + mm * 60 + ss;}
    public static Jam_2511533008 fromSeconds(int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");
        total %=24 * 3600; // wrap 24 jam
        int h = total / 3600; total %= 3600;
        int m = total / 60; int s = total % 60;
        return new Jam_2511533008(h,m,s);
    }
```

Gambar 2.2.1.1 kode Jam_2511533008 1

```java
    //---Fungsional---
    public int compareTo (Jam_2511533008 other) { return Integer.compare(this.toSeconds (), other.toSeconds());}
    public boolean equals(Object o) {
        if (!(o instanceof Jam_2511533008 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }
    public int  hashCode() {return java.util.Objects.hash(hh, mm, ss); }

    //---Aritmatika---
    public Jam_2511533008 plus (Jam_2511533008 other) { return fromSeconds(this.toSeconds() + other.toSeconds()); }
    public Jam_2511533008 minus (Jam_2511533008 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600));}
    public Jam_2511533008 nextSecond() { return fromSeconds(this.toSeconds() + 1);}
    public Jam_2511533008 nextSeconds (int n) { return fromSeconds(this.toSeconds() + Math.max(0, n));}
    public Jam_2511533008 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600));}
    public Jam_2511533008 prevSeconds (int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24*3600));}

    //---Durasi (detik) -- bisa negatif jika this > other? sesuai spesifikasi durasi(JAw, JAkh)
    public static int durasiDetik (Jam_2511533008 jaw, Jam_2511533008 jakh) {return jakh.toSeconds() - jaw.toSeconds();}
    // overriding ke string
    public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }

}
```

Gambar 2.2.1.2 kode Jam_2511533008 2

Penjelasan kode program:

1. Validator

```
//---Validator---
public static boolean isValid(int h, int m, int s) {
    return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
}
```

Gambar 2.2.1.3 Validator

Validator dalam ADT Jam berfungsi dalam memvalidasi apakah nilai jam, menit, dan detik yang di masukkan valid.

2. Konstruktor/kreator

```
//---Konstruktor/Kreator---
public Jam_2511533008(int h, int m, int s) {
    this.hh = h; this.mm = m; this.ss = s;
}
```

Gambar 2.2.1.4 Konstruktor/kreator

Konstruktor berfungsi sebagai kreator. Dengan adanya konstruktor, setiap kali kita membuat objek jam, kita bisa langsung memberikan nilai awal jam, menit, dan detik.

Nama konstruktor harus sama dengan nama kelas yang dibuat.

3. Selektor

```
//---Selektor---
public int getHour() {return hh;}
public int getMinute() {return mm;}
public int getSecond() {return ss;}
```

Gambar 2.2.1.5 Selektor

Selektor adalah method yang digunakan untuk mengambil nilai atribut dari sebuah objek. Pada ADT ini selector di pakai untuk membaca nilai atribut hh (jam), mm (menit), dan ss (detik).

- getHour() → mengembalikan nilai hh (jam)
- getMinute() → mengembalikan nilai mm (menit)
- getSecond() → mengembalikan nilai ss (detik)

4. Mutator

```
//---Mutator---
public void setHour(int h) {this.hh = h; }
public void setMinute(int m) {this.mm = m;}
public void setSecond(int s) {this.ss = s;}
```

Gambar 2.2.1.6 Mutator

Mutator digunakan untuk mengubah nilai atribut dari sebuah objek. Pada ADT ini mutator digunakan untuk memperbarui nilai jam (hh), menit (mm), dan detik (ss).

- SetHour → mengubah nilai atribut hh menjadi h.
- setMinute → mengubah nilai atribut mm menjadi m.
- setSecond → mengubah nilai atribut ss menjadi s.

5. Konversi

```
//---Konversi---
public int toSeconds() { return hh * 3600 + mm * 60 + ss;}
public static Jam_2511533008 fromSeconds(int total) {
    if (total < 0) throw new IllegalArgumentException("detik negatif");
    total %=24 * 3600; // wrap 24 jam
    int h = total / 3600; total %= 3600;
    int m = total / 60; int s = total % 60;
    return new Jam_2511533008(h,m,s);
}
```

Gambar 2.2.1.7 Konversi

- toSeconds () → mengubah jam, menit dan detik menjadi total detik.
- fromSeconds (int total) → mengubah total detik kembali menjadi jam, menit dan detik.

6. Fungsional

```
//---Fungsional---
public int compareTo (Jam_2511533008 other) { return Integer.compare(this.toSeconds (), other.toSeconds());}
public boolean equals(Object o) {
    if (!(o instanceof Jam_2511533008 j)) return false;
    return hh == j.hh && mm == j.mm && ss == j.ss;
}
public int  hashCode() {return java.util.Objects.hash(hh, mm, ss); }
```

Gambar 2.2.1.8 Fungsional

- compareTo (Jam_2511533008 other) digunakan untuk membandingkan dua objek jam dari total detik.
- equals(Object o) mengecek dua objek jam memiliki nilai jam, menit dan detik yang sama.
- hashCode() menghasilkan nilai hash unik berdasarkan kombinasi atribut hh, mm, dan ss.

7. Aritmatika

```
//---Aritmatika---
public Jam_2511533008 plus (Jam_2511533008 other) { return fromSeconds(this.toSeconds() + other.toSeconds()); }
public Jam_2511533008 minus (Jam_2511533008 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600));}
public Jam_2511533008 nextSecond() { return fromSeconds(this.toSeconds() + 1);}
public Jam_2511533008 nextSeconds (int n) { return fromSeconds(this.toSeconds() + Math.max(0, n));}
public Jam_2511533008 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24*3600));}
public Jam_2511533008 prevSeconds (int n) { return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24*3600));}
```

Gambar 2.2.1.9 Aritmatika

- plus (Jam_2511533008 other) menjumlahkan dua objek jam menggunakan total detik lalu hasilnya dikonversi lagi menjadi jam, menit dan detik.
- minus (Jam_2511533008 other) menghitung selisih antar dua jam dalam detik. Math.*floorMod*(this.toSeconds() - other.toSeconds(), 24*3600 memastikan hasil tetap dalam rentang 0-24 jam (wrap around).
- nextSecond () menghasilkan jam baru yang maju sebanyak 1 detik.
- nextSeconds (int n) menghasilkan jam baru yang maju sebanyak n detik.
- prevSecond () menghasilkan jam baru yang mundur sebanyak 1 detik.
- prevSeconds (int n) menghasilkan jam baru yang mundur sebanyak n detik.

8. Durasi (detik)

```
//---Durasi (detik) -- bisa negatif jika this > other? sesuai spesifikasi durasi(JAw, JAkh)
public static int durasiDetik (Jam_2511533008 jaw, Jam_2511533008 jakh) {return jakh.toSeconds() - jaw.toSeconds();}
```

Gambar 2.2.1.10 Durasi (detik)

Menghitung durasi antar waktu.

9. Overriding ke string

```
// overriding ke string
public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss); }
```

Gambar 2.2.1.11 overriding ke string

Dapat menentukan bagaimana objek ditampilkan dalam string.

2.2.2 Membuat Jam Driver 1

1. Membuat class baru dengan nama JamDriver_2511533008.
2. Kode program :

```java
package pekan1_2511533008;

public class JamDriver_2511533008 {
    public static void main (String[] args) {
        Jam_2511533008 a = new Jam_2511533008(23, 59, 50);
        Jam_2511533008 b = new Jam_2511533008(0, 0, 15);
        System.out.println ("a              = " + a);
        System.out.println ("b              = " + b);
        System.out.println ("a+b            = " + a.plus (b));
        System.out.println ("next 20s       = " + a.nextSeconds(20));
        System.out.println ("durasi(a,b)    = " + Jam_2511533008.durasiDetik(a,b));
        System.out.println ("a.compareTo (b)= " + a.compareTo(b));
    }

}
```

Gambar 2.2.2.1 kode JamDriver_2511533008

Penjelasan kode program :

JamDriver digunakan untuk menguji metode yang telah dibuat dalam adt Jam.

Pada kode program ini langsung dimasukkan jam yang akan diuji.

2.2.3 Membuat Jam Driver 2

1. Membuat class baru dengan nama JamDriver2_2511533008.
2. Kode program:

```java
package pekan1_2511533008;
import java.util.Scanner;
public class JamDriver2_2511533008 {
    public static void main (String[] args) {
        Scanner input = new Scanner (System.in);
        System.out.println("=== Program Driver Objek Jam ===");
        // 1. Input Jam pertama
        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511533008 j1 = buatJamDariInput(input);

        //2. Input Jam Kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511533008 j2 = buatJamDariInput(input);

        //3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)        : " + j1.toString());
        System.out.println("Jam 2 (String)        : " + j2.toString());
        System.out.println("Jam 1 dalam detik     : " + j1.toSeconds());
        System.out.println("Jam 1 dalam detik     : " + j2.toSeconds());

        //4. Operasi Relasional (Perbandingan)
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println ("Status              : Jam 1 lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0) {
            System.out.println ("Status              : Jam 1 lebih awal (sebelum) Jam 2");
        } else {
            System.out.println ("Status              : Jam 1 dan Jam 2 sama persis");
        }

        //5. Operasi Aritmatika
        System.out.println ("Durasi (J1 ke J2)          : " + Jam_2511533008.durasiDetik(j1, j2) + " detik");

        Jam_2511533008 jNext = j1.nextSecond();
        System.out.println ("Jam 1 Detik Berikutnya     : " + jNext);
```

Gambar 2.2.2.1 kode JamDriver2_2511533008 1

```java

        Jam_2511533008 jPrev = j1.prevSecond();
        System.out.println ("Jam 1 Detik Sebelumnya      : " + jPrev);

        //6. Operasi Penjumlahan Jam
        Jam_2511533008 jHasilPlus = j1.plus(j2);
        System.out.println ("Hasil J1 : J2               : " + jHasilPlus);

        input.close();
        System.out.println("\nProgram Selesai");
    }
    /**
     * Prosedur pembantu untuk melakukan input dan validasi secara berulang
     * sampai user memasukkan angka yang benar (0...23, 0...59).
     */
    private static Jam_2511533008 buatJamDariInput(Scanner sc) {
        int h,m,s;
        while (true) {
            System.out.print("Masukkan Jam (0-23)    : ");
            h = sc.nextInt();
            System.out.print("Masukkan Menit (0-59)  : ");
            m = sc.nextInt();
            System.out.print("Masukkan Detik (0-29)  : ");
            s = sc.nextInt();

            //Memanggil method static isValid dari kelas Jam
            if (Jam_2511533008.isValid(h, m, s)) {
                return new Jam_2511533008(h, m, s);
            } else {
                System.out.println("(Error) Input tidak valid Silahkan Ulangi.\n");
            }
        }

    }

}
```

Gambar 2.2.2.1 kode JamDriver2_2511533008 2

Penjelasan kode program :

JamDriver2 digunakan untuk menguji metode yang telah dibuat pada Jam dimana user dapat menginputkan jam 1 dan jam 2 untuk di uji.

2.3 Output program

2.3.1 Output Jam Driver

```
Problems   @ Javadoc   Console
<terminated> JamDriver_2511533008 [Java A
a                 = 23:59:50
b                 = 00:00:15
a+b               = 00:00:05
next 20s          = 00:00:10
durasi(a,b)       = -86375
a.compareTo (b)= 1
```

Gambar 2.3.1 output jam driver

2.3.2 Output Jam Driver 2

```
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23)    : 2
Masukkan Menit (0-59)  : 45
Masukkan Detik (0-29)  : 34

--- Input Jam 2 ---
Masukkan Jam (0-23)    : 15
Masukkan Menit (0-59)  : 0
Masukkan Detik (0-29)  : 5

--- Hasil Operasi ---
Jam 1 (String)         : 02:45:34
Jam 2 (String)         : 15:00:05
Jam 1 dalam detik      : 9934
Jam 1 dalam detik      : 54005
Status               : Jam 1 lebih awal (sebelum) Jam 2
Durasi (J1 ke J2)           : 44071 detik
Jam 1 Detik Berikutnya      : 02:45:35
Jam 1 Detik Sebelumnya      : 02:45:33
Hasil J1 : J2               : 17:45:39

Program Selesai
```

Gambar 2.3.2 output jam driver 2

BAB III
PENUTUP

3.1 Kesimpulan

Kesimpulan dari praktikum Struktur Data tentang ADT Jam adalah mahasiswa berhasil memahami bagaimana konsep *Abstract Data Type* dapat digunakan untuk merepresentasikan waktu secara terstruktur. Melalui implementasi kelas Jam, mahasiswa dapat merancang konstruktor, selektor, mutator, serta berbagai operasi seperti validasi, konversi ke detik, perbandingan, aritmatika waktu, durasi, hingga representasi string yang rapi. Praktikum ini juga membuktikan bahwa ADT Jam tidak hanya menyimpan data, tetapi mampu melakukan manipulasi dan perhitungan waktu yang relevan dengan kasus nyata, seperti penjadwalan atau simulasi durasi kegiatan. Dengan demikian, praktikum ini memperkuat keterampilan analisis, logika, dan pemahaman abstraksi data, sekaligus menghubungkan teori struktur data dengan implementasi nyata dalam bahasa pemrograman Java.

DAFTAR PUSTAKA

[1] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Wiley, 2014.

[2] I. Lim, *Diktat Struktur Data*. Bandung: Institut Teknologi Bandung, 2008.

[3] P. J. Deitel and H. M. Deitel, *Java: How to Program (Early Objects)*, 11th ed. Pearson Education, 2017.

[4] Oracle, *The Java™ Tutorials*. [Online]. Available: https://docs.oracle.com/javase/tutorial/. [Accessed: Mar. 31, 2026].